

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«КРОС-ПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри АІС Бельська В. Ю.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 6

Тема: Java Collections Framework. Stream API.

Мета: реалізувати консольний додаток, що дозволяє вирішити поставлені завдання з використанням стандартних колекцій та потоків Stream API.

Порядок виконання роботи:

При реалізації кожного із заданих потрібно обов'язково обрати клас із Java Collections Framework, який підходить для рішень задач. Для обробки запитів потрібно обов'язково використовувати Stream API.

Завдання 1. Необхідно розробити програму, яка дозволить зберігати інформацію про авторизації користувачів. Кожному користувачеві відповідає пара логін та пароль. При старті програма відображає меню:

- Додати нового користувача;
- Видалити існуючого користувача;
- Перевірити чи існує користувач;
- Змінити логін існуючого користувача;
- Змінити пароль користувача;

Завдання 3. Створіть додаток, що емулює чергу у популярне кафе. Відвідувачі кафе приходять та потрапляють у чергу, якщо немає вільного місця. Коли столик в кафе стає вільним, перший відвідувач з черги займає його. Якщо приходить відвідувач, який резервував столик на певний час, він отримує зарезервований столик позачергово.

Завдання 5. За допомогою класів та зв'язків між ними описати вулицю міста.

На вулиці мають бути присутніми будинки різного типу. Наприклад, житлові будинки, магазини, лікарні, школи. Передбачити тестову ініціалізацію кожного об'єкта та вулиці загалом.

Кожен будинок повинен мати певну адресу в межах вулиці. Крім того, кожен тип будинку повинен мати свою сукупність полів. Наприклад, має бути тип магазину та перелік відділів у ньому (наприклад приватний маленький магазин повинен мати 1 відділ, супермаркет 5 відділів), кількість мешканців у житловому будинку, кількість учнів та рівень акредитації у школі (загальноосвітня, гімназія, ліцей тощо).

Кількість учнів у школі прив'язати до рівня акредитації та випадково обирати у певному діапазоні.

Передбачити можливість встановлення адреси кожного об'єкта як через конструктор, і через сеттер. Передбачити можливість додавання нового будинку на вулицю та видалення будинку. Також передбачити віртуальний метод, який прийматиме рядок та на його основі встановлюватиме поля об'єкта. Зробити висновок інформації про кожну оселю в консоль. Зробити виведення інформації вулицею в консоль. Зробити спосіб, який для випадково обраного житлового будинку знаходить у заданій околиці (певну кількість будинків від адреси будинку) усі магазини, що мають відділ заданого типу.

Генерацію тестового покриття зробити за допомогою статичної фабрики.

Взаємодія з користувачем зробити через меню, при цьому передбачити окремий клас.

У роботі використовувати інтерфейси, абстрактні класи (за потреби) та перерахування. За потреби використовувати механізм обробки винятків.

Завдання 1:

UserManager.java:

```
import java.util.*;

class UserManager {

    private Map<String, String> users = new HashMap<>();

    // додати
    public void addUser(String login, String password) {
        users.put(login, password);
    }

    // видалити
    public void removeUser(String login) {
        users.remove(login);
    }

    // перевірка (Stream API)
    public boolean exists(String login) {
        return users.keySet()
            .stream()
            .anyMatch(l -> l.equals(login));
    }
}
```

```

    }

    // змінити логін
    public void changeLogin(String oldLogin, String newLogin) {
        if (users.containsKey(oldLogin)) {
            String pass = users.remove(oldLogin);
            users.put(newLogin, pass);
        }
    }

    // змінити пароль
    public void changePassword(String login, String newPass) {
        users.computeIfPresent(login, (k, v) -> newPass);
    }

    public void printAll() {
        users.forEach((l, p) -> System.out.println(l + ":" + p));
    }
}

```

Main.java:

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        UserManager um = new UserManager();
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {

                System.out.println("\n1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід");
                int choice = sc.nextInt();
                sc.nextLine();

                switch (choice) {

                    case 1:
                        System.out.print("Логін: ");
                        String login = sc.nextLine();
                        System.out.print("Пароль: ");
                        String pass = sc.nextLine();
                        um.addUser(login, pass);
                        break;

                    case 2:
                        System.out.print("Логін: ");
                        um.removeUser(sc.nextLine());
                        break;

```

```

        case 3:
            System.out.print("Логін: ");
            System.out.println(um.exists(sc.nextLine()) ? "Існує" :
"Немає");

            break;

        case 4:
            System.out.print("Старий логін: ");
            String oldL = sc.nextLine();
            System.out.print("Новий логін: ");
            String newL = sc.nextLine();
            um.changeLogin(oldL, newL);
            break;

        case 5:
            System.out.print("Логін: ");
            String l = sc.nextLine();
            System.out.print("Новий пароль: ");
            um.changePassword(l, sc.nextLine());
            break;

        case 6:
            um.printAll();
            break;

        case 0:
            return;

    }

}

}

}

```

```

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
1
Логін: ihorpolynko

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
1
Логін: ihorpolynko
Пароль: 1235
1
Логін: ihorpolynko
Пароль: 1235
Пароль: 1235

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
3
Логін: ihorpolynko
Існує

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
4
Старий логін: ihorpolynko
Новий логін: ihor

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
5
Логін: ihor
Новий пароль: 123

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
6
ihor:123

1-Додати 2-Видалити 3-Перевірка 4-Змінити логін 5-Змінити пароль 6-Вивід 0-Вихід
0

```

Рисунок 6.1 – Результат роботи скрипту завдання 1

Завдання 3:

Visitor.java:

```

class Visitor {
    String name;
    int reserveTime; // время резерва (например, в "шагах" или минутах)

    public Visitor(String name, int reserveTime) {
        this.name = name;
        this.reserveTime = reserveTime;
    }
}

```

CafeQueue.java:

```

import java.util.Deque;
import java.util.LinkedList;

```

```

import java.util.Optional;
import java.util.Comparator;

class CafeQueue {

    private Deque<Visitor> queue = new LinkedList<>();

    // звичайний відвідувач
    public void addVisitor(String name) {
        queue.addLast(new Visitor(name, -1)); // -1 = немає резерву
    }

    // резерв
    public void addReserved(String name, int time) {
        for (Visitor v : queue) {
            if (v.name.equals(name)) {
                v.reserveTime = time;
                System.out.println("Оновлено резерв для: " + name);
                return;
            }
        }
        queue.addLast(new Visitor(name, time));
    }

    // звільнився столик
    // serve at a given current time (minutes since 00:00)
    public void serve(int timeNow) {

        // шукаємо резерви, час яких вже настав (reserveTime != -1 && reserveTime
        <= timeNow)
        Optional<Visitor> reservedNow = queue.stream()
            .filter(v -> v.reserveTime >= 0 && v.reserveTime <= timeNow)
            .min(Comparator.comparingInt(v -> v.reserveTime));

        if (reservedNow.isPresent()) {
            Visitor v = reservedNow.get();
            queue.remove(v);
            System.out.println("Сів (резерв): " + v.name);
            return;
        }

        if (!queue.isEmpty()) {
            System.out.println("Сів: " + queue.pollFirst().name);
        } else {
            System.out.println("Черега пуста");
        }
    }
}

```

Main.java:

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        CafeQueue cafe = new CafeQueue();
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1-Додати відвідувача 2-Додати резерв 3-  
Вільний столик 4-Вихід");
                int choice = sc.nextInt();
                sc.nextLine();

                switch (choice) {
                    case 1:
                        System.out.print("Ім'я відвідувача: ");
                        cafe.addVisitor(sc.nextLine());
                        break;
                    case 2:
                        System.out.print("Ім'я (резерв): ");
                        String name = sc.nextLine();
                        System.out.print("Час резерву (число або HH:MM): ");
                        String timeStr = sc.nextLine().trim();
                        int time = parseTimeToMinutes(timeStr);
                        if (time < 0) {
                            System.out.println("Невірний формат часу резерву.  
Використано -1 (без резерву).");
                        }
                        cafe.addReserved(name, time);
                        break;
                    case 3:
                        System.out.print("Поточний час (HH:MM або година): ");
                        String nowStr = sc.nextLine().trim();
                        int now = parseTimeToMinutes(nowStr);
                        if (now < 0) {
                            System.out.println("Невірний формат поточного часу.  
Використано 0.");
                        }
                        now = 0;
                        }
                        cafe.serve(now);
                        break;
                    case 4:
                        return;
                }
            }
        }

        private static int parseTimeToMinutes(String s) {
```



```

        if (s == null || s.isEmpty()) return -1;
        s = s.trim();
        if (s.contains(":")) {
            String[] parts = s.split(":");
            if (parts.length >= 2) {
                try {
                    int hh = Integer.parseInt(parts[0].trim());
                    int mm = Integer.parseInt(parts[1].trim());
                    if (hh >= 0 && hh < 24 && mm >= 0 && mm < 60) return hh * 60
+ mm;

                } catch (NumberFormatException e) {
                    return -1;
                }
            }
            return -1;
        }

        String digits = s.replaceAll("\\D+", "");
        if (digits.isEmpty()) return -1;

        try {
            if (digits.length() <= 2) {
                int hh = Integer.parseInt(digits);
                if (hh >= 0 && hh < 24) return hh * 60;
                return -1;
            } else if (digits.length() == 3 || digits.length() == 4) {
                int hh = Integer.parseInt(digits.substring(0, digits.length() -
2));

                int mm = Integer.parseInt(digits.substring(digits.length() - 2));
                if (hh >= 0 && hh < 24 && mm >= 0 && mm < 60) return hh * 60 +
mm;

                return -1;
            } else {
                return Integer.parseInt(digits);
            }
        } catch (NumberFormatException ex) {
            return -1;
        }
    }
}

```

```
1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
1
Ім'я відвідувача: Ihor

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
1
Ім'я відвідувача: Alina

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
1
Ім'я відвідувача: Max

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
2
Ім'я (резерв): Ihor
Час резерву (число або HH:MM): 12:00
Оновлено резерв для: Ihor

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
2
Ім'я (резерв): Alina
Час резерву (число або HH:MM): 11:30
Оновлено резерв для: Alina

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
3
Поточний час (HH:MM або година): 11:45
Сів (резерв): Alina

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
3
Поточний час (HH:MM або година): 12:00
Сів (резерв): Ihor

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
3
Поточний час (HH:MM або година): 12:30
Сів: Max

1-Додати відвідувача 2-Додати резерв 3-Вільний столик 4-Вихід
3
Поточний час (HH:MM або година): 13:00
Черга пуста
```

Рисунок 6.2 – Результат роботи скрипту завдання 3

Завдання 5:

Building.java:

```
abstract class Building {

    protected String address;

    public Building(String address) {
        this.address = address;
    }
}
```

```

    }

    public void setAddress(String address) {
        this.address = address;
    }

    public abstract void print();
}

```

School.java:

```

// Акредитація школи
enum SchoolType {
    GENERAL, GYMNASIUM, LYCEUM
}

class School extends Building {

    private int students;
    private SchoolType type;

    public School(String address, SchoolType type, int students) {
        super(address);
        this.type = type;
        this.students = students;
    }

    public int getStudents() { return students; }
    public SchoolType getType() { return type; }

    public void setFromString(String str) {
        String[] parts = str.split(",");
        this.type = SchoolType.valueOf(parts[0]);
        this.students = Integer.parseInt(parts[1]);
    }

    @Override
    public void print() {
        System.out.println("Школа " + address + " [" + type + "] учнів: " +
students);
    }
}

```

Hospital.java:

```

class Hospital extends Building {

    private int beds;

    public Hospital(String address, int beds) {
        super(address);
    }
}

```

```

        this.beds = beds;
    }

    @Override
    public void print() {
        System.out.println("Лікарня " + address + " ліжок: " + beds);
    }
}

```

House.java:

```

class House extends Building {

    private int residents;

    public House(String address, int residents) {
        super(address);
        this.residents = residents;
    }

    public int getResidents() {
        return residents;
    }

    // Встановлення через рядок
    public void setFromString(String str) {
        // наприклад, "20" – кількість мешканців
        this.residents = Integer.parseInt(str);
    }

    @Override
    public void print() {
        System.out.println("Будинок " + address + " мешканці: " + residents);
    }
}

```

Shop.java:

```

import java.util.Arrays;
import java.util.List;

class Shop extends Building {

    private List<String> departments;
    private boolean isSupermarket;

    public Shop(String address, List<String> dep) {
        super(address);
        this.departments = dep;
        this.isSupermarket = dep.size() > 1;
    }
}

```

```

    }

    public List<String> getDepartments() { return departments; }
    public boolean isSupermarket() { return isSupermarket; }

    @Override
    public void print() {
        String type = isSupermarket ? "Супермаркет" : "Приватний магазин";
        System.out.println(type + " " + address + " відділи: " + departments);
    }

    // Віртуальний метод для встановлення відділів через рядок
    public void setFromString(String str) {
        // наприклад, рядок "Їжа,Одяг"
        departments = Arrays.asList(str.split(","));
        isSupermarket = departments.size() > 1;
    }
}

```

Street.java:

```

import java.util.*;
import java.text.Normalizer;
import java.util.Locale;

class Street {

    private List<Building> buildings = new ArrayList<>();

    public void add(Building b) {
        buildings.add(b);
    }

    public void remove(String address) {
        buildings.removeIf(b -> b.address.equals(address));
    }

    public void printAll() {
        buildings.forEach(Building::print);
    }

    public List<Building> getBuildings() {
        return buildings;
    }

    public void findShops(String department) {
        String normDept = normalize(department);

        buildings.stream()
            .filter(b -> b instanceof Shop)
            .map(b -> (Shop) b)

```

```

        .filter(s -> s.getDepartments().stream()
            .anyMatch(d -> normalize(d).equals(normDept)))
        .forEach(Shop::print);
    }

    // знайти магазини поруч з будинком
    public void findNearbyShops(House house, int range, String department) {
        int index = buildings.indexOf(house);
        if (index == -1)
            return;

        int from = Math.max(0, index - range);
        int to = Math.min(buildings.size(), index + range + 1);

        String normDept = normalize(department);

        buildings.subList(from, to).stream()
            .filter(b -> b instanceof Shop)
            .map(b -> (Shop) b)
            .filter(s -> s.getDepartments().stream()
                .anyMatch(d -> normalize(d).equals(normDept)))
            .forEach(Shop::print);
    }

    private String normalize(String s) {
        if (s == null) return "";
        String n = Normalizer.normalize(s, Normalizer.Form.NFKC);
        n = n.replace('\u00A0', ' ').trim();
        return n.toLowerCase(Locale.ROOT);
    }
}

```

Factory.java:

```

import java.util.*;

class Factory {

    static Random r = new Random();

    static House randomHouse() {
        return new House("A" + r.nextInt(100), r.nextInt(50) + 1);
    }

    static Shop randomShop() {
        List<String> possibleDeps = Arrays.asList("Food", "Clothes", "Tech");

        int count = r.nextBoolean() ? 1 : r.nextInt(5) + 1;
        List<String> deps = new ArrayList<>();
        for (int i = 0; i < count; i++) {
            deps.add(possibleDeps.get(r.nextInt(possibleDeps.size())));
        }
    }
}

```

```

    }
    return new Shop("B" + r.nextInt(100), deps);
}

static School randomSchool() {
    SchoolType type =
SchoolType.values()[r.nextInt(SchoolType.values().length)];
    int students;
    switch (type) {
        case GENERAL:
            students = 200 + r.nextInt(100);
            break;
        case GYMNASIUM:
            students = 150 + r.nextInt(50);
            break;
        case LYCEUM:
            students = 100 + r.nextInt(50);
            break;
        default:
            students = 100;
    }
    return new School("S" + r.nextInt(100), type, students);
}

static Hospital randomHospital() {
    return new Hospital("H" + r.nextInt(100), 50 + r.nextInt(50));
}
}

```

Main.java:

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Street street = new Street();
        Scanner sc = new Scanner(System.in);

        // тестова генерація
        for (int i = 0; i < 3; i++) {
            street.add(Factory.randomHouse());
            street.add(Factory.randomShop());
            street.add(Factory.randomSchool());
            street.add(Factory.randomHospital());
        }

        boolean running = true;
        while (running) {
            System.out.println("\n1 - Показати всю вулицю");
            System.out.println("2 - Знайти магазини за відділом");

```

```

System.out.println("3 - Додати будинок");
System.out.println("4 - Видалити будинок за адресою");
System.out.println("5 - Знайти магазини поруч з будинком");
System.out.println("0 - Вихід");

int choice = sc.nextInt();
sc.nextLine(); // consume newline

switch (choice) {
    case 1:
        street.printAll();
        break;
    case 2:
        System.out.print("Введіть відділ: ");
        String dep = sc.nextLine();
        street.findShops(dep);
        break;
    case 3:
        System.out.println("1-House 2-Shop 3-School 4-Hospital");
        int type = sc.nextInt();
        sc.nextLine();
        switch (type) {
            case 1:
                street.add(Factory.randomHouse());
                break;
            case 2:
                street.add(Factory.randomShop());
                break;
            case 3:
                street.add(Factory.randomSchool());
                break;
            case 4:
                street.add(Factory.randomHospital());
                break;
        }
        break;
    case 4:
        System.out.print("Введіть адресу: ");
        street.remove(sc.nextLine());
        break;
    case 5:
        System.out.print("Введіть адресу будинку: ");
        String addr = sc.nextLine();

        System.out.print("Введіть відділ: ");
        String depNearby = sc.nextLine();

        House selectedHouse = null;

        for (Building b : street.getBuildings()) {
            if (b instanceof House &&
b.address.equalsIgnoreCase(addr)) {

```



```

        selectedHouse = (House) b;
        break;
    }
}

if (selectedHouse == null) {
    System.out.println("Будинок не знайдено");
    break;
}

System.out.println("Обраний будинок:");
selectedHouse.print();

System.out.println("Магазини поруч:");
street.findNearbyShops(selectedHouse, 2, depNearby);
break;
case 0:
    running = false;
    break;
}
}
sc.close();
}
}

```

```

1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
1
Будинок A49 мешканці: 21
Супермаркет B87 відділи: [Food, Clothes, Food]
Школа S52 [GENERAL] учнів: 284
Лікарня H68 ліжок: 73
Будинок A29 мешканці: 20
Приватний магазин B37 відділи: [Tech]
Школа S38 [GYMNASIUM] учнів: 170
Лікарня H56 ліжок: 95
Будинок A70 мешканці: 33
Супермаркет B9 відділи: [Tech, Clothes, Clothes]
Школа S33 [GYMNASIUM] учнів: 189
Лікарня H16 ліжок: 60

1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
2
Введіть відділ: Food
Супермаркет B87 відділи: [Food, Clothes, Food]

```

Рисунок 6.3 – Результат роботи скрипту завдання 5

```
1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
3
1-House 2-Shop 3-School 4-Hospital
3

1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
4
Введіть адресу: A29

1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
1
Будинок A49 мешканці: 21
Супермаркет B87 відділи: [Food, Clothes, Food]
Школа S52 [GENERAL] учнів: 284
Лікарня H68 ліжок: 73
Приватний магазин B37 відділи: [Tech]
Школа S38 [GYMNASIUM] учнів: 170
Лікарня H56 ліжок: 95
Будинок A70 мешканці: 33
Супермаркет B9 відділи: [Tech, Clothes, Clothes]
Школа S33 [GYMNASIUM] учнів: 189
Лікарня H16 ліжок: 60
Школа S23 [GYMNASIUM] учнів: 151
```

Рисунок 6.4 – Продовження результату роботи скрипту завдання 5

```
1 - Показати всю вулицю
2 - Знайти магазини за відділом
3 - Додати будинок
4 - Видалити будинок за адресою
5 - Знайти магазини поруч з будинком
0 - Вихід
5
Введіть адресу будинку: A70
Введіть відділ: Clothes
Обраний будинок:
Будинок A70 мешканці: 33
Магазини поруч:
Супермаркет B9 відділи: [Tech, Clothes, Clothes]
```

Рисунок 6.5 – Продовження результату роботи скрипту завдання 5

Висновок: на цій лабораторній роботі ми реалізували консольний додаток, що дозволяє вирішити поставлені завдання з використанням стандартних колекцій та потоків Stream API